



Análisis Sintáctico

DISEÑO E IMPLEMENTACIÓN DE UN LENGUAJE

v0.1.0

1. Introducción	1
2. Gramática	2
2.1. Precedencia y Asociatividad	3
3. Semántica	4
3.1. Gramática Atribuida	5
3.2. Árbol de Sintaxis Abstracta	7
4. Conflictos	10
4.1. Desplazamiento/Reducción	10
4.2. Reducción/Reducción	11
5. Bibliografía	11

1. Introducción

Nuevamente, sea $G = \langle \Sigma, N, \Pi, S \rangle$ una gramática libre de contexto, donde Σ es el alfabeto, N es el conjunto de símbolos no-terminales, Π es el conjunto de producciones y S es el símbolo no-terminal inicial, se define un **analizador sintáctico** (también conocido como *parser*), como aquel cuyo objetivo principal es transformar una secuencia de símbolos del alfabeto Σ en uno o varios *árboles de sintaxis abstracta*¹ (AST).

La cantidad de árboles generados dependerá del tipo de analizador y de la secuencia analizada en su entrada, de modo tal que si la entrada pertenece al lenguaje descrito por G , se emite un árbol por cada posible secuencia de derivaciones que genere dicha entrada (si G no es

¹ Es importante recalcar el hecho de que un analizador sintáctico genera árboles de sintaxis debido a que en este contexto el documento se restringe a gramáticas libres de contexto, pero efectivamente para gramáticas de mayor jerarquía el concepto de árbol como estructura de datos es inaplicable.

ambigua, siempre se emite un único árbol a lo sumo), pero un conjunto vacío en caso de que la entrada no pertenezca a $L(G)$.

Para las siguientes secciones, el analizador sintáctico utilizado será Bison (un generador de analizadores LALR(1)), pero las ideas y conceptos son generales a otros analizadores. Este analizador solo emite un único AST en caso de tener éxito al analizar la secuencia de entrada, debido a que el generador es inferior a un DPDA (*Deterministic Pushdown Automaton*), y por ende, no admite gramáticas ni lenguajes ambiguos.

2. Gramática

Todo lenguaje conlleva la definición de su sintaxis y su correspondiente semántica. Para estructurar la sintaxis del lenguaje se emplea una gramática que define las construcciones válidas que el lenguaje ofrece. Estas construcciones se deben especificar mediante las 4 componentes de la gramática.

Para definir la lista de tokens que definen el alfabeto Σ de la gramática mediante el generador Bison, se debe utilizar la siguiente directiva:

```
%token <semanticValueName> TOKEN_NAME
```

Donde `TOKEN_NAME` es el nombre del token que se emplea en las acciones del respectivo escáner generado por Flex, tales como `ADD`, `MUL` u `OPEN_PARENTHESIS`. Debido a que estos tokens conforman el alfabeto, representan el conjunto de **símbolos terminales** de la gramática.

No obstante, para definir el conjunto de **símbolos no-terminales**, se emplea una directiva similar:

```
%type <semanticValueName> nonTerminalName
```

De forma análoga, `nonTerminalName` es el nombre de un símbolo no-terminal disponible para utilizar en la definición de las reglas de producción de la gramática G (conjunto Π). Nótese que por convención, se emplea `SCREAMING_SNAKE_CASE` para los terminales, pero `camelCase` para los no-terminales.

Tanto terminales como no-terminales acarrean un atributo o metadato que representa su **valor semántico** (cuyo nombre se indica mediante el parámetro `semanticValueName` en ambos casos). La definición y uso de ellos será detallada en la **Sección § 3: Semántica**.

Con ambos conjuntos, ya es posible especificar la gramática, en este caso, un ejemplo de programa que admite expresiones aritméticas sobre constantes enteras:

```
program: expression
      ;

expression: expression ADD expression
          | expression DIV expression
          | expression MUL expression
          | expression SUB expression
```

```
| factor
;

factor: OPEN_PARENTHESIS expression CLOSE_PARENTHESIS
| constant
;

constant: INTEGER
;
```

Nótese que el símbolo “;” (semicolon), indica el final de una secuencia de producciones que poseen el mismo no-terminal del lado izquierdo, mientras que “:” (colon) reemplaza el clásico símbolo “→” de una regla de derivación. En el caso de que exista la necesidad de utilizar una regla de producción cuyo lado derecho sea λ , se deberá dejar el espacio vacío luego del “|” (pipe), o bien, utilizar la directiva `%empty` en su lugar.

En este ejemplo, es fácil ver que un programa en este lenguaje es en realidad una expresión, y que la misma se segrega en una combinación de factores y constantes que solo pueden ser de tipo enteras, interactuando mediante los operadores básicos de la aritmética (suma, resta, multiplicación y división), posiblemente agrupando ciertas subexpresiones mediante paréntesis.

En la gramática, el símbolo `INTEGER` debería representar un único elemento del alfabeto, y en este caso evidentemente es así, pero nótese que debido a la integración de Bison con Flex, es sabido que dicho token puede generarse desde infinitos lexemas posibles, específicamente aquellos que representan secuencias finitas de dígitos, pero arbitrariamente largas².

2.1. Precedencia y Asociatividad

La gramática de expresiones aritméticas expuesta en la sección anterior no impone ningún orden natural sobre las operaciones admitidas. Es evidente que, jerárquicamente, una expresión es un concepto más general que un factor, y que un factor es más general que una constante, pero nada se dice acerca de la relación que existe entre la suma y la multiplicación, o entre la suma y la resta.

En cualquier lenguaje de programación (e incluso en aquellos que no tienen por objetivo el diseño de aplicaciones pero que constan de un grupo de operadores), es deseable definir un orden en particular para dos situaciones particulares: (I) la combinación de operadores del mismo tipo, y (II) las de diferente tipo.

Cuando se dispone de una secuencia de operaciones, donde el operador es el mismo en sucesivas aplicaciones (situación I):

$$A + B + C + D + \dots$$

es relevante hablar de **asociatividad**, es decir, del agrupamiento natural que posee dicho operador sobre sus operandos. La asociatividad se puede dar *por izquierda* si:

² Es posible, y en general deseable, que el largo de dichas secuencias se limite de alguna manera mediante las expresiones regulares de Flex, en particular si dicha secuencia, por ejemplo, será traducida y almacenada en una variable de tipo `int` de C, que no admite más que lo que quepa en 4 bytes, al menos en la mayoría de plataformas hasta la fecha.

$$A + B + C + D + \dots = (((A + B) + C) + D) + \dots),$$

por derecha:

$$A + B + C + D + \dots = (A + (B + (C + (D + \dots)))),$$

o como es el caso del operador de suma, por ambas.

Por otro lado, en los casos en que el operador se encuentre en una secuencia junto a otros de diferente tipo (situación II), se hablará de **precedencia**, ya que en estos casos no es relevante el agrupamiento natural de cada operador (podría haber una única ocurrencia de cada uno dentro de una misma expresión, con lo cual las reglas de asociatividad no aplican), sino más bien el orden en el cual dichas operaciones se deben ejecutar para evaluar o computar el valor de la expresión final.

La precedencia usualmente se especifica como una simple jerarquización de operadores (ver por ejemplo, [C Operator Precedence](#)), donde aquellos con mayor precedencia se ejecutan o computan antes que aquellos con menor precedencia. Dentro de clases o niveles de la misma jerarquía, los operadores se ejecutan en el mismo orden en el que se encuentran o escanean en la secuencia de entrada (de izquierda a derecha).

Para especificar la asociatividad en Bison, se disponen 2 (dos) directivas adicionales: **%left** y **%right**, que indican la asociatividad a izquierda o a derecha respectivamente. Para indicar la precedencia, no obstante, se utiliza el mismo orden en el cual dichas directivas fueron redactadas:

```
%left ADD SUB
%left MUL DIV
```

donde el hecho de que la asociatividad de MUL y DIV se haya definido luego de la de los operadores ADD y SUB, implica que estos últimos poseen **menor precedencia**, con lo cual, una expresión que involucre símbolos de ambos niveles de jerarquía, priorizará el cómputo de la multiplicación o división **antes** que la de la suma y la resta, lo que es consistente con el comportamiento de dichos operadores en la aritmética.

Este mecanismo simplifica enormemente la definición de la gramática, ya que de otro modo, las reglas de producción deberán ser las que definan explícitamente la precedencia y asociatividad naturalmente como un efecto secundario del proceso de derivación de la secuencia de entrada.

3. Semántica

Una gramática definida en Bison no produce ningún efecto adicional más allá de reconocer que la secuencia de entrada es generada por la gramática o lo que es igual, pertenece al lenguaje que dicha gramática genera.

No obstante, un compilador en general aplica transformaciones y validaciones adicionales sobre la secuencia de entrada. Para ello, de alguna manera debe poder representar la secuencia

de entrada mediante alguna estructura de datos conveniente (y no simplemente como un *stream* de tokens), y permitir el agregado de información adicional.

El objetivo entonces es lograr llevar el poder de una gramática libre de contexto más allá de sus capacidades (a un nivel más alto en la Jerarquía de Chomsky), y para ello se hará uso del concepto de gramática atribuida.

3.1. Gramática Atribuida

La teoría de este modelo matemático desarrollado por D. E. Knuth (1968) será desarrollada en otros documentos, pero a modo simplificado, Bison considera que cada token puede acarrear un atributo o metadato que representa de alguna manera la semántica del mismo. Este valor semántico puede ser de cualquier tipo de dato, y se puede asignar tanto a símbolos terminales como no-terminales.

Para definir la lista de posibles valores semánticos disponibles dentro del analizador Bison, se debe utilizar una directiva especial denominada **%union**:

```
%union {  
    /** Terminals. */  
  
    signed int integer;  
    TokenLabel token;  
  
    /** Non-terminals. */  
  
    Constant * constant;  
    Expression * expression;  
    Factor * factor;  
    Program * program;  
}
```

En este caso, un extracto del repositorio de base, específicamente obtenido del documento [BisonGrammar.y](#), que define una estructura de datos compuesta por varios campos de diferente tipo.

Recordando que en el lenguaje C es posible definir estructuras de datos mediante **struct** y **union**³, donde **union** especifica un grupo de campos que se encuentran superpuestos en memoria, efectivamente esta definición provee un valor semántico **polimórfico**, es decir, que puede comportarse o interpretarse de diferentes maneras, según que campo del mismo se utilice.

De este modo, asignar un 0 (cero) al valor semántico de una instancia X del tipo de esta unión, se podrá interpretar como un número entero en caso de ser accedida mediante `X.integer`, o como un puntero a NULL en caso de utilizar `X.constant` o `X.factor`, entre otros.

¿Cómo se indica para cada terminal y no-terminal el tipo de dato del valor semántico (atributo o metadato), que acarrea? Utilizando las directivas **%token** y **%type** presentadas en la **Sección § 2: Gramática**, específicamente utilizando el parámetro `semanticValueName`. Por ejemplo, las siguientes definiciones:

³ [Union declaration - cppreference.com](#)

```

/** Terminals. */
%token <integer> INTEGER
%token <token> ADD
...

/** Non-terminals. */
%type <constant> constant
...

```

indican que los terminales `INTEGER` y `ADD` poseen un valor semántico de tipo `int` y `Token` respectivamente, ya que los campos denominados `integer` y `token` efectivamente poseen este tipo de dato en la unión. Por otro lado, el no-terminal `constant`, posee un valor semántico de tipo `Constant*`, y es accedido mediante un campo que se denomina igual que el símbolo.

¿Cómo se asigna un valor semántico a un terminal provisto por Flex? Utilizando el contexto provisto por la función `createLexicalAnalyzerContext()`, el cual provee una copia única del campo `semanticValue` que justamente es del tipo de dato de esta unión definida en Bison. Este campo se genera para cada terminal, y por ende cada uno de ellos puede almacenar sus propios metadatos sin afectar a las demás ocurrencias, incluso del mismo token.

Finalmente, para asignar el valor semántico de un no-terminal en Bison, es necesario utilizar las reglas de producción:

```

...
expression: expression ADD expression          { $$ = f($1, $3); }
          | ...
          ;

factor: OPEN_PARENTHESIS expression CLOSE_PARENTHESIS { $$ = g($2); }
      | constant                                       { $$ = h($1); }
      ;

constant: INTEGER                                  { $$ = $1; }
      ;

```

Para cada regla de la forma $A \rightarrow B(1) \dots B(n)$, `$$` representa el valor semántico del símbolo `A`, mientras que `$k`, representa el valor semántico del símbolo `B(k)` (con $1 \leq k \leq n$). De este modo, la acción asociada a la expresión de suma, le asigna a la expresión resultante (no-terminal `expression` del lado izquierdo de la regla), el valor semántico `$$`, obtenido luego de retornar de la función `f($1, $3)`, donde `$1` y `$3` efectivamente representan a su vez el valor semántico de las expresiones involucradas en la suma (operando izquierdo y derecho, respectivamente).

Nótese que para computar `f($1, $3)`, no se hizo uso del valor `$2`, ya que no es relevante en este caso utilizar el atributo dentro del token `ADD`. En situaciones como las de la acción asociada a la regla `constant` $\rightarrow$ `INTEGER`, el valor semántico `$1` simplemente se propaga hacia `$$`.

Debido a que por las directivas `%token` y `%type`, se especificó que para el no-terminal `constant` su tipo de la unión debía ser `Constant*`, mientras que para el token `INTEGER` debía ser `int`, está claro que la asignación `$$ = $1` es incompatible en sus tipos de datos, y por ende producirá un error al ser compilado. Esto implica que dicha asignación es imposible *verbatim*, y que deberá transformarse el valor de `$1` en uno que quepa dentro de `$$`, por ejemplo, aplicando un *wrapping* con alguna estructura adicional, como `Constant`:

```
typedef struct Constant Constant;

struct Constant {
    int value;
};
```

En este caso la asignación es simple, porque solo involucra un tipo de dato único, sin embargo nótese que el valor semántico de `factor` puede ser conformado desde una `expression`, o desde una `constant`. Para estos casos, es preferible realizar alguna especie de “polimorfismo manual”, aprovechando el uso de los enumerados, estructuras y uniones de C:

```
typedef struct FactorType FactorType;

typedef struct Factor Factor;

enum FactorType {
    CONSTANT,
    EXPRESSION
};

struct Factor {
    union {
        Constant * constant;
        Expression * expression;
    };
    FactorType type;
};
```

Ahora un `Factor` puede acarrear una constante o expresión (sin desperdiciar memoria gracias al uso de la unión). Por otro lado, puede ser manipulado sin ambigüedad gracias al enumerado `FactorType`, que permite identificar explícitamente qué campo de la unión es el que efectivamente se está transportando dentro de una instancia de esta estructura⁴.

Finalmente, en ciertas ocasiones el uso de las etiquetas `$1`, `$2`, `$3`, ..., puede dificultar la lectura de la gramática. Para ello, Bison permite el uso de *named references* (entre corchetes):

```
expression: expression[left] ADD expression[right]    { $$ = f($left, $right); }
          | ...
          ;
```

3.2. Árbol de Sintaxis Abstracta

El primer paso está completo: la gramática transporta y computa nuevos atributos o metadatos a lo largo del análisis sintáctico pero, ¿qué sucede si fases posteriores a dicho análisis

⁴ Efectivamente este mecanismo idiomático que simula el polimorfismo es útil en C específicamente porque dicho lenguaje no soporta el polimorfismo de tipos de forma nativa. En otros lenguajes más avanzados, el uso de tipos polimórficos (como en Haskell), o el polimorfismo de clases y el uso de interfaces sería mucho más adecuado (como en Java o TypeScript). Esto simplemente es un *hack* al lenguaje.

deben efectuar alguna transformación sobre la secuencia de entrada y por ende deben conocer y navegar la estructura sintáctica de la misma? Para resolver este problema se hace evidente la necesidad de mapear la secuencia de tokens inicial en la entrada a una estructura de datos que provea ambas características: (I) debe ser navegable, y (II) debe ser representativa de la sintaxis.

La estructura que provee ambas características se denomina **Árbol de Sintaxis Abstracta** (AST), y debe ser un árbol debido a que una gramática libre de contexto impone naturalmente dicha topología: para cada producción de la forma $A \rightarrow B(1) \dots B(n)$, A es un nodo padre, y $B(k)$ es alguno de sus n hijos.

Es factible imaginar que debido a las siguientes cuestiones:

- (I). Cada símbolo terminal y no-terminal de una gramática puede transportar un valor semántico.
- (II). Cada no-terminal efectivamente se encontrará del lado izquierdo de una producción y por ende será el padre de varios nodos.
- (III). Cada terminal conformará un nodo hoja en este árbol (porque nunca estará del lado izquierdo de una regla).

se podría utilizar el valor semántico de los símbolos no-terminales para transportar los nodos que conforman este AST. Ahora bien, ya que Bison es un analizador sintáctico de tipo LALR(1), el analizador ejecuta las reducciones de cada regla de forma **ascendente**, es decir, el AST subyacente que la gramática define naturalmente para cierta entrada se genera en **post-orden**.

Esto significa que al ejecutar una reducción sobre una regla de producción, se ejecuta su acción (el código de C encerrado entre `{ ... }`), pero solo cuando ya se han ejecutado las acciones asociadas a todos los símbolos del lado derecho de dicha producción. Convenientemente, este orden de ejecución del algoritmo subyacente de Bison permite que la creación de un nodo en el AST para el no-terminal a la izquierda de $A \rightarrow B(1) \dots B(n)$, se pueda construir y enlazar directamente a sus n nodos hijo $B(k)$, donde cada nodo $B(k)$ está completamente construido, incluso si estos nodos son raíces de otros subárboles dentro del AST.

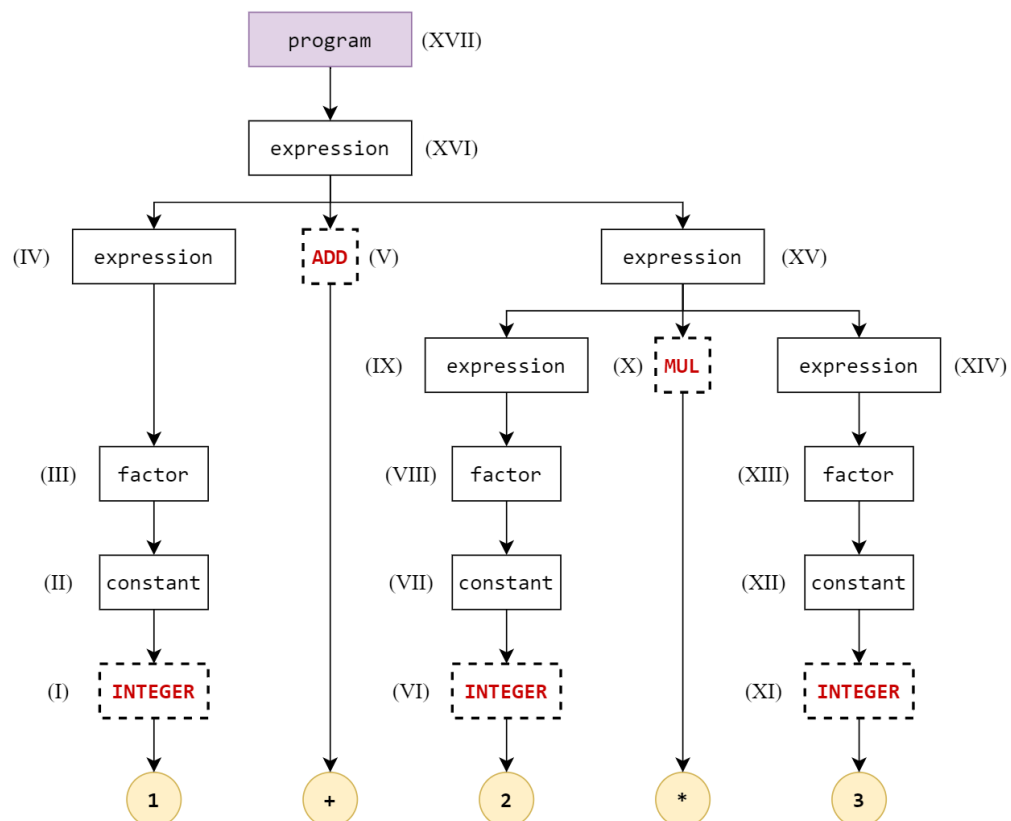
Para ejemplificar el orden de ejecución de estas acciones, se aplicará el análisis sobre la expresión $1 + 2 * 3$ utilizando el repositorio base [Flex-Bison-Compiler](#):

```
[DEBUG][SyntacticAnalyzer] Parsing...
[DEBUG][FlexActions] IntegerLexemeAction: 1 (context = 0, length = 1, line = 1)
[DEBUG][BisonActions] IntegerConstantSemanticAction
[DEBUG][BisonActions] ConstantFactorSemanticAction
[DEBUG][BisonActions] FactorExpressionSemanticAction
[DEBUG][FlexActions] ArithmeticOperatorLexemeAction: + (context = 0, length = 1, line = 1)
[DEBUG][FlexActions] IntegerLexemeAction: 2 (context = 0, length = 1, line = 1)
[DEBUG][BisonActions] IntegerConstantSemanticAction
[DEBUG][BisonActions] ConstantFactorSemanticAction
[DEBUG][BisonActions] FactorExpressionSemanticAction
[DEBUG][FlexActions] ArithmeticOperatorLexemeAction: * (context = 0, length = 1, line = 1)
[DEBUG][FlexActions] IntegerLexemeAction: 3 (context = 0, length = 1, line = 1)
[DEBUG][BisonActions] IntegerConstantSemanticAction
[DEBUG][BisonActions] ConstantFactorSemanticAction
[DEBUG][BisonActions] FactorExpressionSemanticAction
[DEBUG][BisonActions] ArithmeticExpressionSemanticAction
[DEBUG][BisonActions] ArithmeticExpressionSemanticAction
[DEBUG][BisonActions] ExpressionProgramSemanticAction
[DEBUG][SyntacticAnalyzer] Parsing is done.
```


Del log anterior reportado durante la ejecución de las acciones es posible ver que:

- (I). Flex identifica y genera el atributo para un token INTEGER de valor semántico 1.
- (II). Bison aplica la reducción $\text{constant} \rightarrow \text{INTEGER}$.
- (III). Bison aplica la reducción $\text{factor} \rightarrow \text{constant}$.
- (IV). Bison aplica la reducción $\text{expression} \rightarrow \text{factor}$.
- (V). Flex identifica y genera el atributo para un token ADD.
- (VI). Flex identifica y genera el atributo para un token INTEGER de valor semántico 2.
- (VII). Bison aplica la reducción $\text{constant} \rightarrow \text{INTEGER}$ (esta vez para el valor semántico 2).
- (VIII). Bison aplica la reducción $\text{factor} \rightarrow \text{constant}$.
- (IX). Bison aplica la reducción $\text{expression} \rightarrow \text{factor}$.
- (X). Flex identifica y genera el atributo para un token MUL.
- (XI). Flex identifica y genera el atributo para un token INTEGER de valor semántico 3.
- (XII). Bison aplica la reducción $\text{constant} \rightarrow \text{INTEGER}$ (esta vez para el valor semántico 3).
- (XIII). Bison aplica la reducción $\text{factor} \rightarrow \text{constant}$.
- (XIV). Bison aplica la reducción $\text{expression} \rightarrow \text{factor}$.
- (XV). Bison aplica la reducción $\text{expression} \rightarrow \text{expression MUL expression}$.
- (XVI). Bison aplica la reducción $\text{expression} \rightarrow \text{expression ADD expression}$.
- (XVII). Bison aplica la reducción $\text{program} \rightarrow \text{expression}$.

Como AST, la secuencia de operaciones se ejecuta en el orden esperado (post-orden, ascendente):



En otras palabras, al ejecutar por ejemplo, la reducción:

`expression → expression ADD expression,`

los valores semánticos **\$1** y **\$3** contendrán los nodos del AST para cada expresión del lado derecho de esta producción, es decir, los nodos raíz de los subárboles de cada expresión asociada a cada operando de la suma. Por otro lado, con este par de subárboles se espera que la acción asociada construya el nodo del no-terminal a la izquierda de la regla, y que asigne dicho valor semántico a **\$\$**, dando por finalizada la construcción de esta parte del AST.

La aplicación sistemática de este proceso culmina con la creación del nodo raíz `program`, que representa toda la estructura sintáctica de la secuencia de entrada original, y la única raíz del AST completo.

4. Conflictos

Debido a que Bison tiene una capacidad de análisis sintáctico LALR(1), ciertas gramáticas libres de contexto no podrán producir un AST como corresponde, ya sea porque dichas gramáticas son ambiguas, o porque el lenguaje que generan requiere capacidades no-determinísticas (como es el caso del lenguaje de los palíndromos, que es no-determinístico, libre de contexto y no-ambiguo). Cuando la gramática excede las capacidades del analizador, este produce uno o más conflictos.

4.1. Desplazamiento/Reducción

El primer tipo de conflicto se produce en general cuando no existe un orden claro de precedencia entre ciertas construcciones de la gramática, tales como es el caso de las expresiones aritméticas definidas en la **Sección § 2: Gramática**. Si la precedencia no fuera especificada mediante las directivas **%left** y **%right**, las 4 (cuatro) operaciones matemáticas podrían ejecutarse en cualquier orden, y por ende el AST producido podría incurrir en múltiples variantes posibles.

En el contexto de un analizador, un **desplazamiento** (*shift*) implica consumir un símbolo adicional de la secuencia de entrada, mientras que una **reducción** (*reduce*) indica la ejecución de una regla de producción determinada. Por lo tanto, un conflicto desplazamiento/reducción (*shift/reduce*), demuestra que el analizador sintáctico se encuentra frente a la posibilidad de efectuar ambas operaciones, situación bajo la cual decide por defecto aplicar el desplazamiento pero, no obstante, sin conocer los efectos que esto representa sobre la semántica final del AST generado. Por este motivo, **cualquier conflicto presente y detectado por Bison, debe ser apropiadamente eliminado.**

4.2. Reducción/Reducción

En este caso, el conflicto implica la posibilidad de aplicar dos o más reducciones diferentes, es decir, la posibilidad de activar 2 (dos) o más reglas de producción para la misma secuencia de entrada. En estos casos, Bison ejecutará la reducción que se definió primero en la gramática, pero de nuevo, sin conocer el impacto semántico que esto produce sobre la compilación final de la secuencia de entrada, motivo por el cual debe eliminarse por completo al igual que un conflicto *shift/reduce*.

La eliminación de conflictos de este tipo en general involucra la reescritura de parte de la gramática. Es común generar este tipo de conflictos por el uso excesivo de producciones anulables (aquellas que utilizan el símbolo λ), o por construcciones que recursivamente permiten derivar las mismas estructuras sintácticas mediante diferentes caminos (diferentes secuencias de derivación).

Tanto para conflictos *shift/reduce* como para *reduce/reduce*, Bison posee un mecanismo que produce contra-ejemplos, simplificando de este modo, no solo la detección temprana de los mismos, sino también la visualización de la estructura sintáctica particular que destraba dicho comportamiento (estructura que el mismo analizador imprime por consola).

5. Bibliografía

- (I). Free Software Foundation, Inc. (2024) Bison 3.8.1 (accedido el 2024-05-06).
 - Sección § 3.3.2: Empty Rules**
 - Sección § 3.4: Defining Language Semantics**
 - Sección § 3.6: Named References**
 - Sección § 5.2: Shift/Reduce Conflicts**
 - Sección § 5.3: Operator Precedence**
 - Sección § 5.6: Reduce/Reduce Conflicts**
 - Sección § 8.1: Generation of Counterexamples**
- (II). Levine, J. R. (2009) [flex & bison](#). O'Reilly Media, Inc.
- (III). Knuth, D. E. (1968) [Semantics of Context-Free Languages](#). Mathematical Systems Theory, 2 (2), 127-145. <https://doi.org/10.1007/BF01692511>